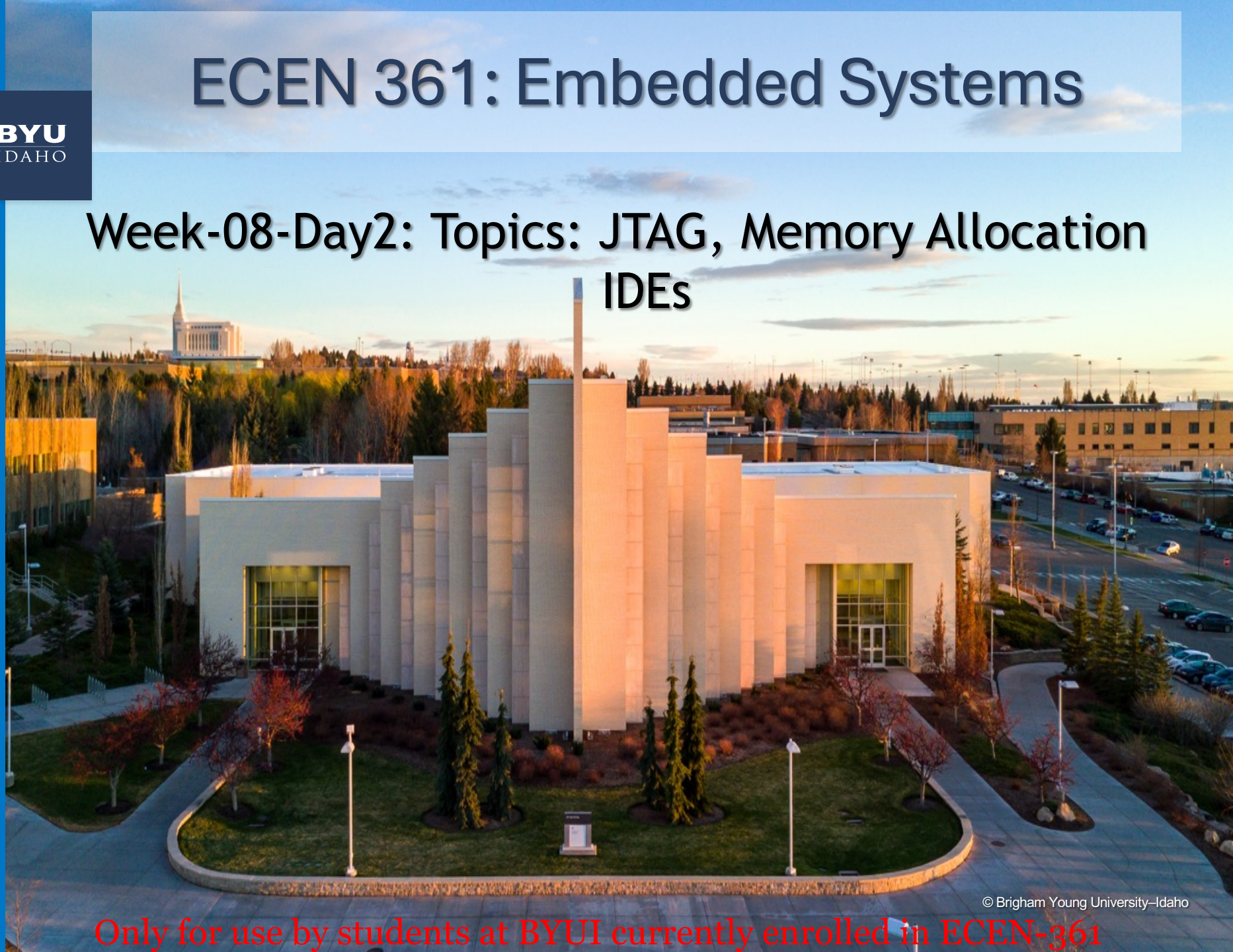
ECEN 361: Embedded Systems

BYU IDAHO

Week-08-Day2: Topics: JTAG, Memory Allocation IDEs



© Brigham Young University–Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Topics Today

- Reminder of Metrics
- JTAG
- Memory Allocation in
  - Bare Metal
  - FreeRTOS
- IDEs

BYU IDAHO

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Final Project Metrics. ("End in Mind")

- At the 'Creative Works Showcase'
- Demonstration by each group to include:
  - Working project (whatever 'wor
  - Informational poster
    (free printer in STC 239 -wide
  - Maybe 'area experts' on differen
  - Formal & informal short presenta
- Grading for the entire pro
  Showcase:
  - 40% on engineering concept, con
  - 40% on execution (build, operation,)
  - 20% on professionalism in present

6-8 professional Engineers as commenters.

Bro. Watson collects and assigns final project grade

Only for use by students at BYUI currently enrolled in ECEN-361

Time for Your Project to Review ?

Goal – Is our team going to be ready?

- Did, Going to do, Blockers
- How much time is left per each task

- Decide on the 'backlog' of things to be done
  - How long is each item predicted to take?
  - Do we need to cut out something? Add help?
- Can we predict success? (blockers, long tent-poles, …)

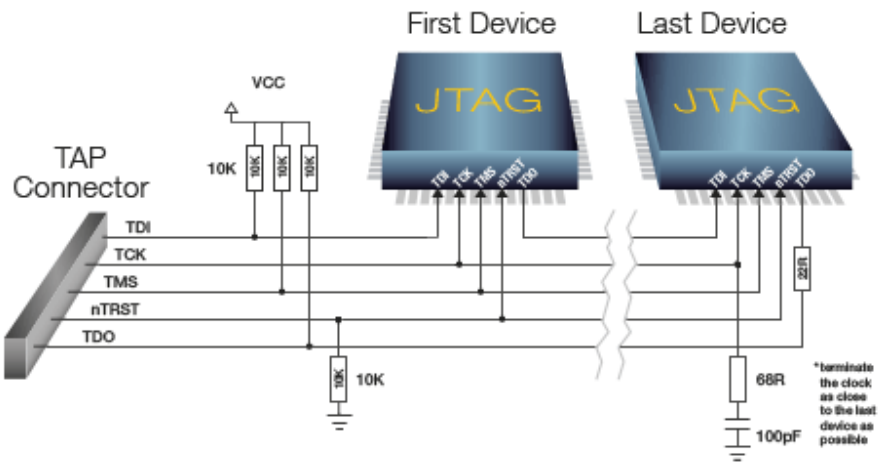
BYU IDAHO

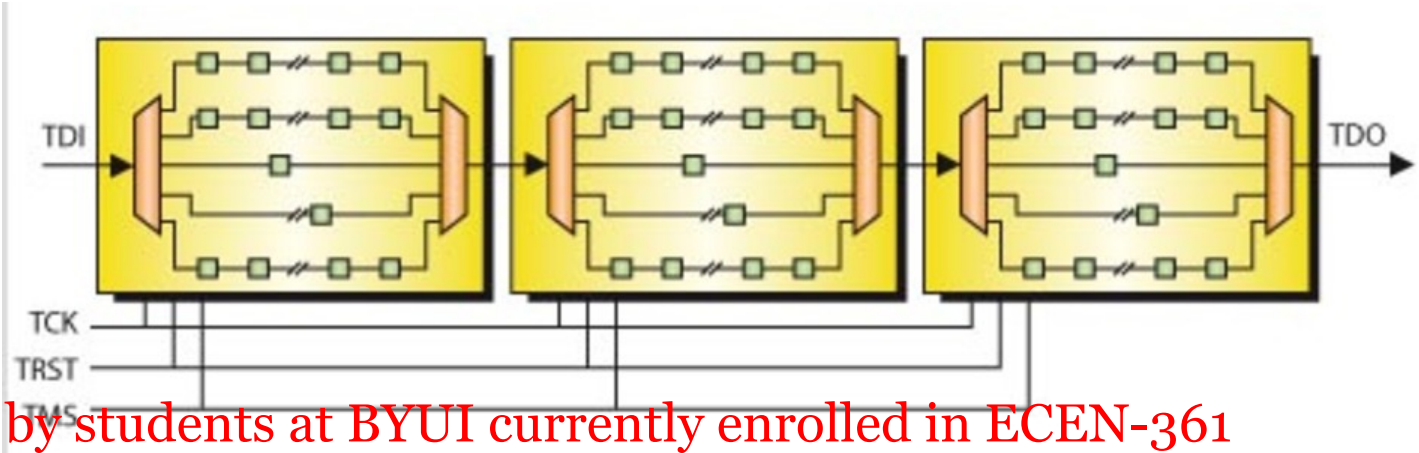
© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Serial Test, Programming, Debug

- Not possible to physically connect to all pins on a device
- Serial "JTAG" connects from chip-to-chip OR block-to -block

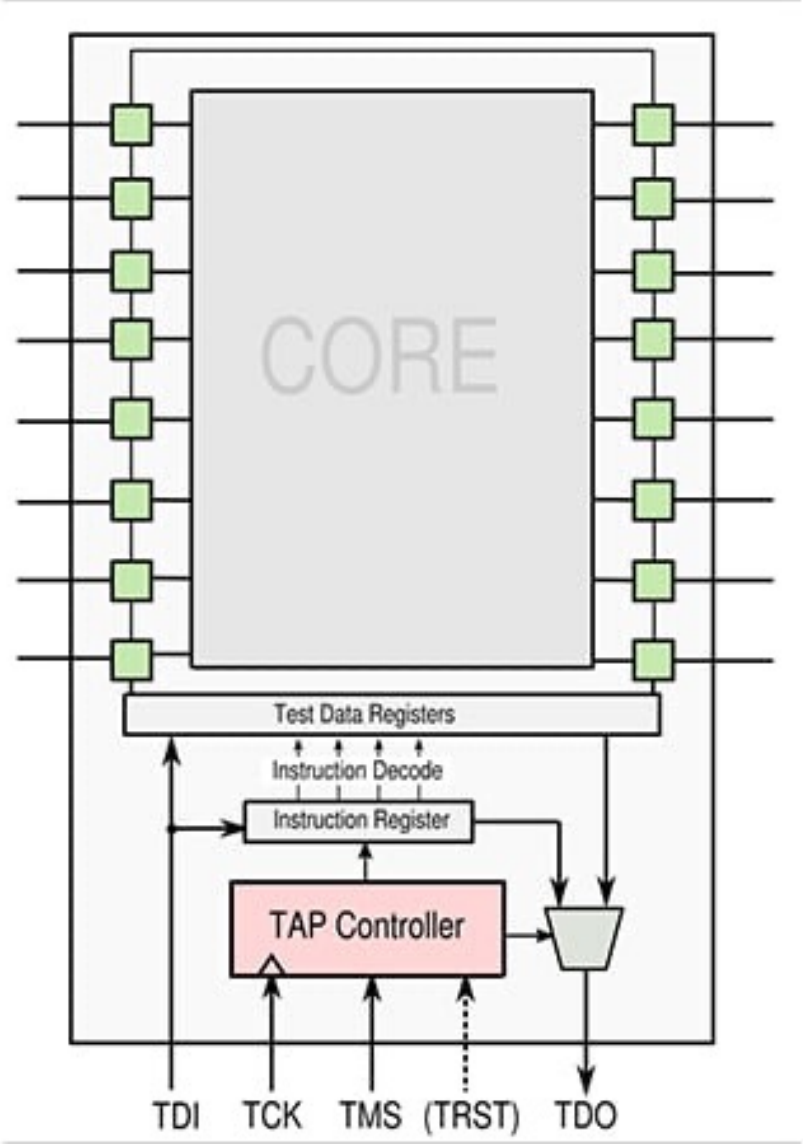




BYU IDAHO
Only for use by students at BYUI currently enrolled in ECEN-361

JTAG interface for uC internals

- Need access to the internals to:
  - Debug
  - Single Step
  - Dump contents of memory, registers

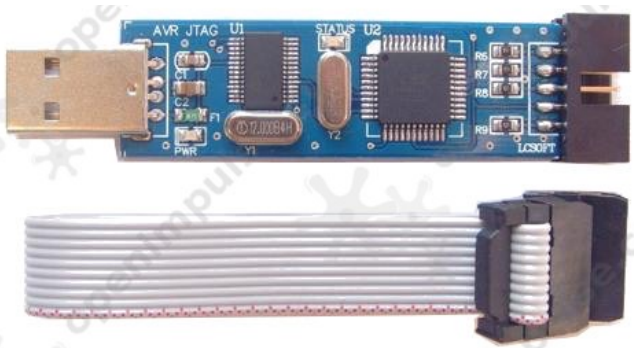


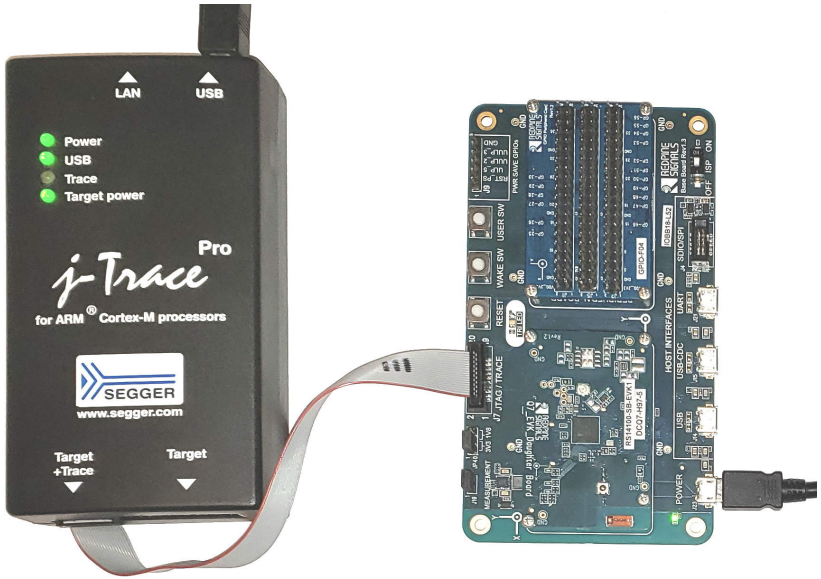
© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

JTAG for Debug

- All uControllers (and chips) have a JTAG port
- JTAG controllers allow :
  - Start, Stop, Breakpoints, Memory, Register, …
- IDEs (CCS, etc.) talk to the JTAG controller



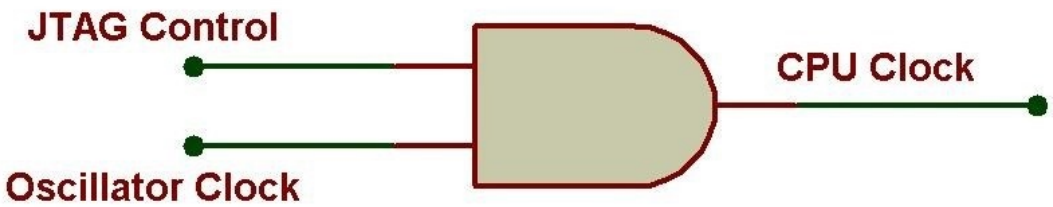


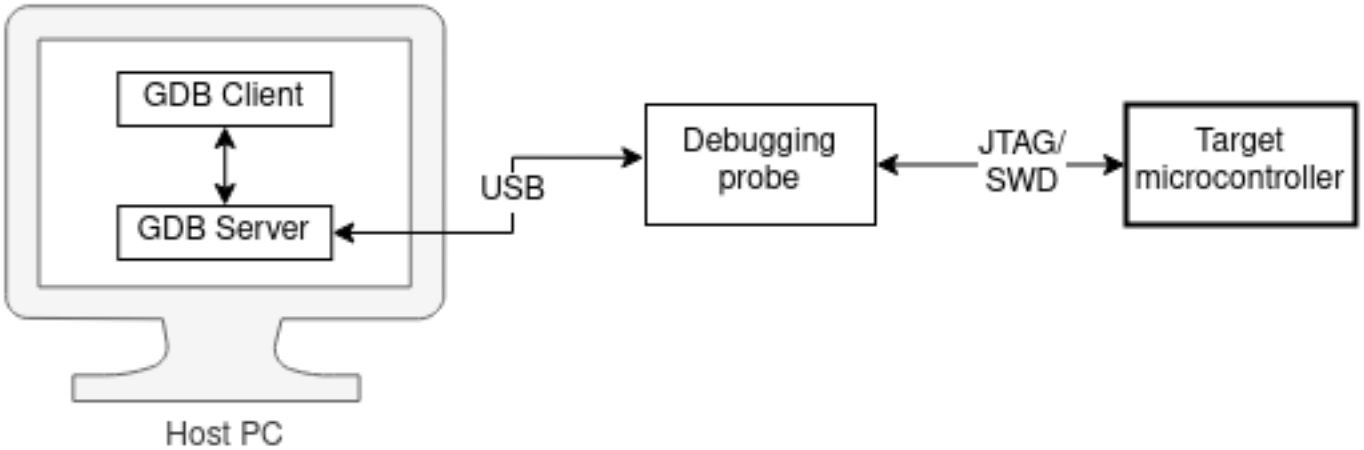
© Brigham Young University - Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Debug Probe / Firmware

- Stop Clock(s)
- S/W talks to Memory Elements on Microcontroller

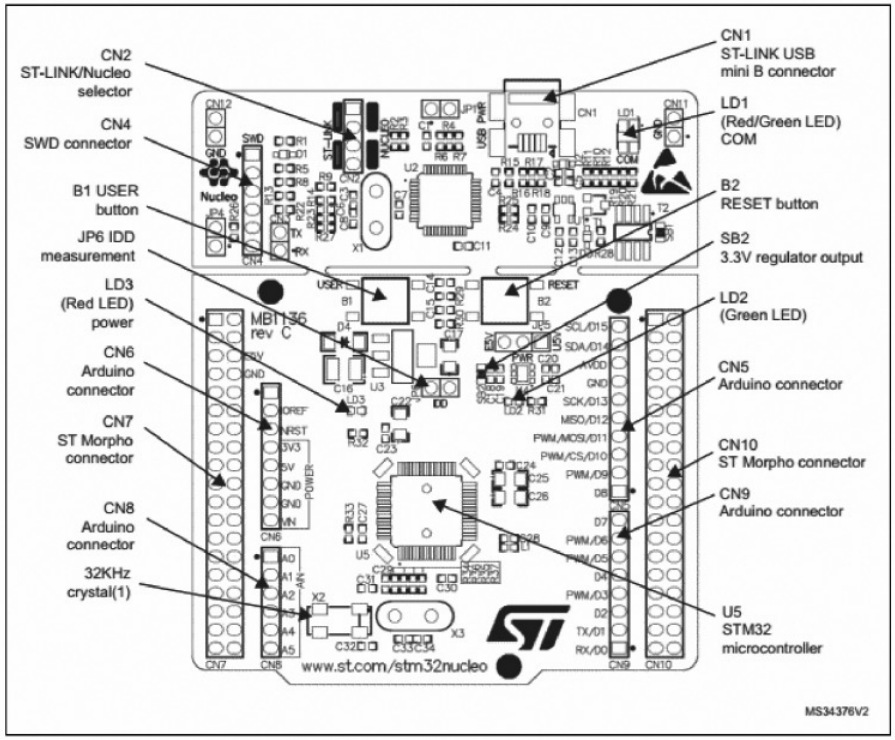




BYU IDAHO
Only for use by students at BYUI currently enrolled in ECEN-361

STM32's JTAG

- When the Debug Controls the Clock:
  - Timers
  - Interrupts
  - FreeRTOS SYSCLK

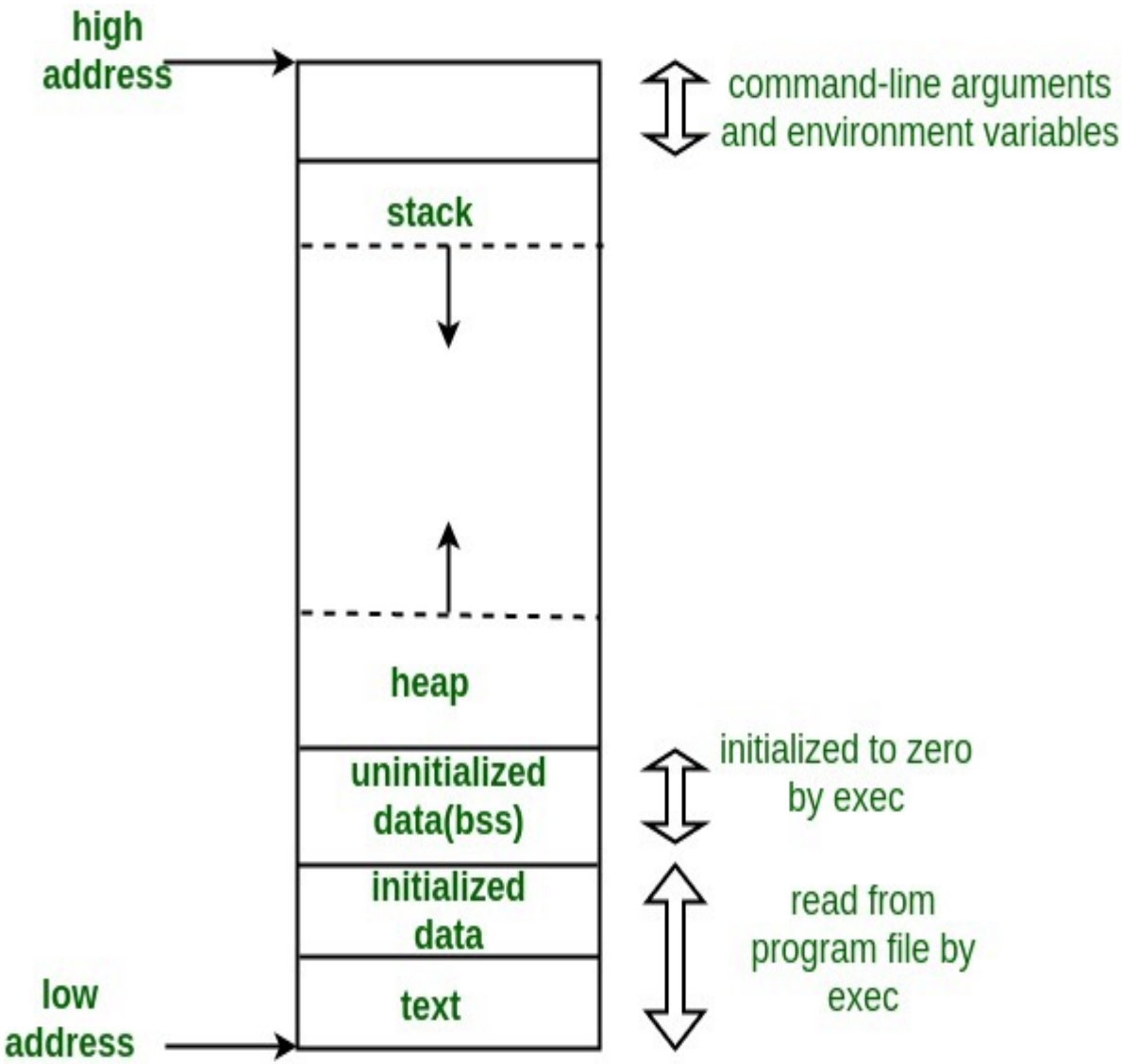


© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Use of Memory in C : “BareMetal”

- Heap:
- Managed by:
  - Malloc
  - realloc,
  - free

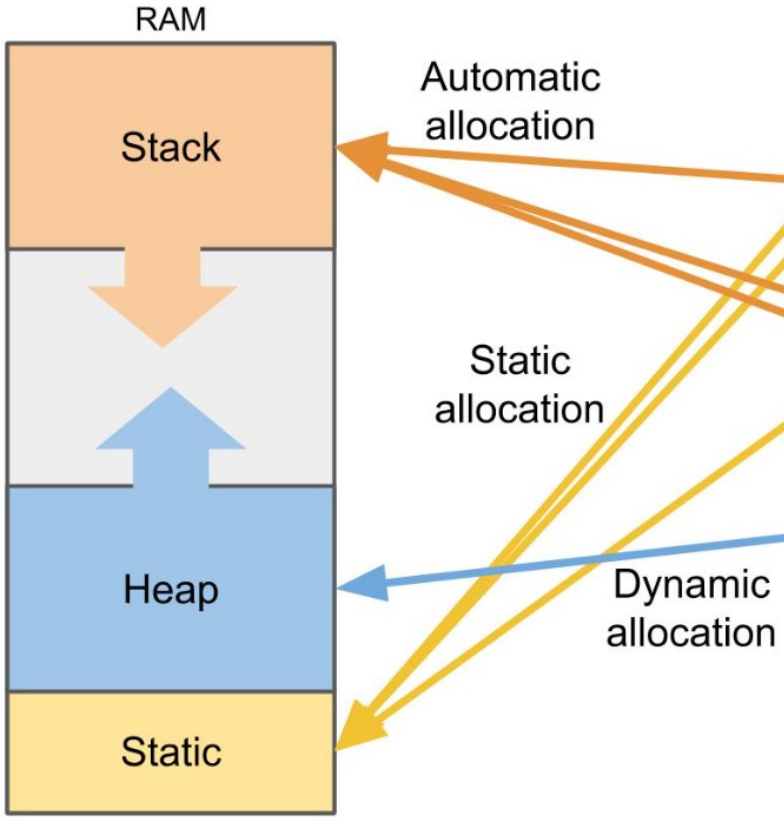


© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Typical Memory Allocation w/Code

Memory Allocation




```c
int global_variable;
static int static_variable;

void someFunction(int some_argument) {

    static int local_static_variable;
    int local_variable;
    int *dynamic_ptr;

    dynamic_ptr = (int*)malloc(32 * sizeof(int));

    // Your function does something here

    free(dynamic_ptr);
}
```


© Brigham Young University—Idaho

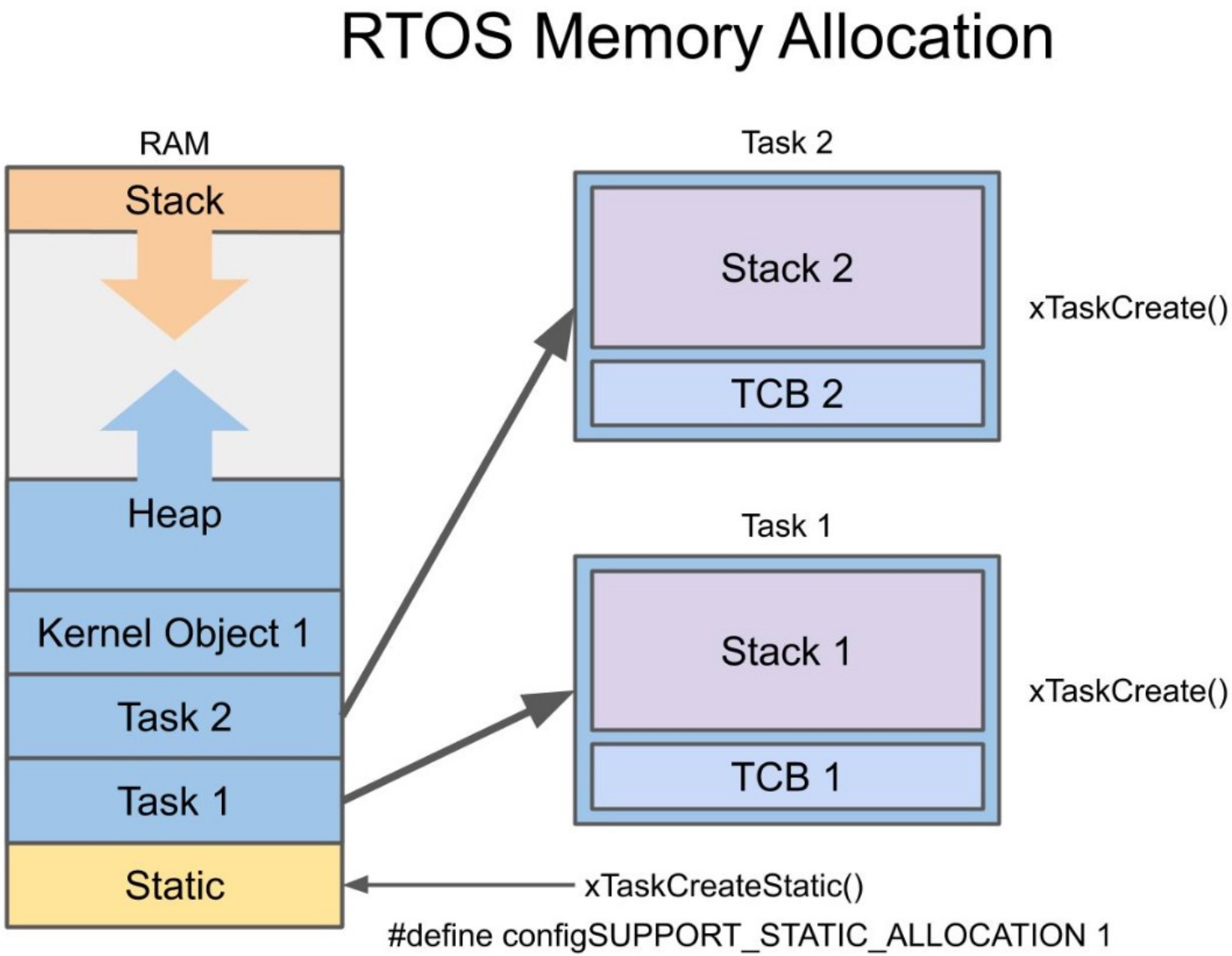
Only for use by students at BYUI currently enrolled in ECEN-361

Memory Allocation in an RTOS

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Allocation in an RTOS (FreeRTOS)



© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Re-stating

- Each task has its own stack
- We define the size at task creation
- The stack stores:
  - Local variables,
  - Function call data,
  - Return addresses.
- Access to local variables (on local stack) is limited to the context
- If a process has allocated ("malloc") a block of storage --- has a pointer

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

Storage reserved by a process

Any Process can create a block of memory on the heap (just like bare-metal)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Allocate memory for an array of 100 integers
    int *array = (int *)malloc(100 * sizeof(int));

    // Assign each element to be 2 times the index
    for (int i = 0; i < 100; i++) {
        array[i] = 2 * i;
    }
    // Free the allocated memory
    free(array);
    return 0;
}
```

But, multi-task problems!!

1. Two+ processes ask for malloc() at the same time?
2. Process can be deleted – before memory is freed

BYU IDAHO

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

”Thread-safe” FreeRTOS calls for memory

```c
#include "FreeRTOS.h"
#include "task.h"  // Required for pvPortMalloc and vPortFree in FreeRTOS

void someTaskFunction(void *pvParameters) {
    // Allocate memory using pvPortMalloc
    int *array = (int *)pvPortMalloc(100 * sizeof(int));

    // Check if allocation was successful (optional in this example)
    if (array != NULL) {
        // Use the allocated memory
        for (int i = 0; i < 100; i++) {
            array[i] = i * 2;
        }
        // Free the allocated memory when done
        vPortFree(array);
    }
    // End of task
    vTaskDelete(NULL);
}
```

Using the RTOS calls `pvPortMalloc(), vPortFree()`

- Guaranteed to not interfere
- Guaranteed to do clean-up on termination even without free()

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

RTOS must know how much memory

At the time of creation - memory is allocated from heap

GUI:
(words)

Code:
(bytes)

*ECEN-361-STM32-Lab-04-FreeRTOS-Solution.ioc

ECEN-361-STM32-Lab-04-FreeRTOS-Solution.ioc - Pinout & Configuration

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Software Packs | Pinout

Categories | A->Z

AIROC-Wi-Fi-Bluetooth-S.
FATFS
FP-ATR-SIGFOX1
FP-SNS-FLIGHT1
FP-SNS-MOTENV1
FP-SNS-MOTENVWB1
FP-SNS-SMARTAG2
FP-SNS-STBOX1
FREERTOS
I-CUBE-Cesium
I-CUBE-FS-RTOS
I-CUBE-ITTIADB
I-CUBE-Mongoose
I-CUBE-UNISONRTOS
I-CUBE-embOS
I-CUBE-wolfMQTT
I-CUBE-wolfSSH
I-CUBE-wolfSSL
I-CUBE-wolfTPM
I-Cube-SoM-uGOAL
TOUCHSENSING

FREERTOS Mode and Configuration

Mode

Interface CMSIS_V2

Configuration

Reset Configuration

Tasks and Queues | Timers and Semaphores | Mutexes | Events | FreeRTOS Heap Usage

Config parameters | Include parameters | Advanced settings | User Constants

Tasks

Task Name	Priority	Stack Size (Words)	Entry Function	Code Generati...	Parameter	Allocation	Buffer Name	Control Blo...
defaultTask	osPriorityNormal	128	StartDefaultTask	Default	NULL	Dynamic	NULL	NULL
D3_Task	osPriorityLow	128	StartD3_Task	Default	NULL	Dynamic	NULL	NULL
D4_Task	osPriorityLow	128	StartD4_Task	Default	NULL	Dynamic	NULL	NULL
SevenSeg_Task	osPriorityLow	128	StartSevenSeg_Task	Default	NULL	Dynamic	NULL	NULL

Add | Delete

Queues

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block Name

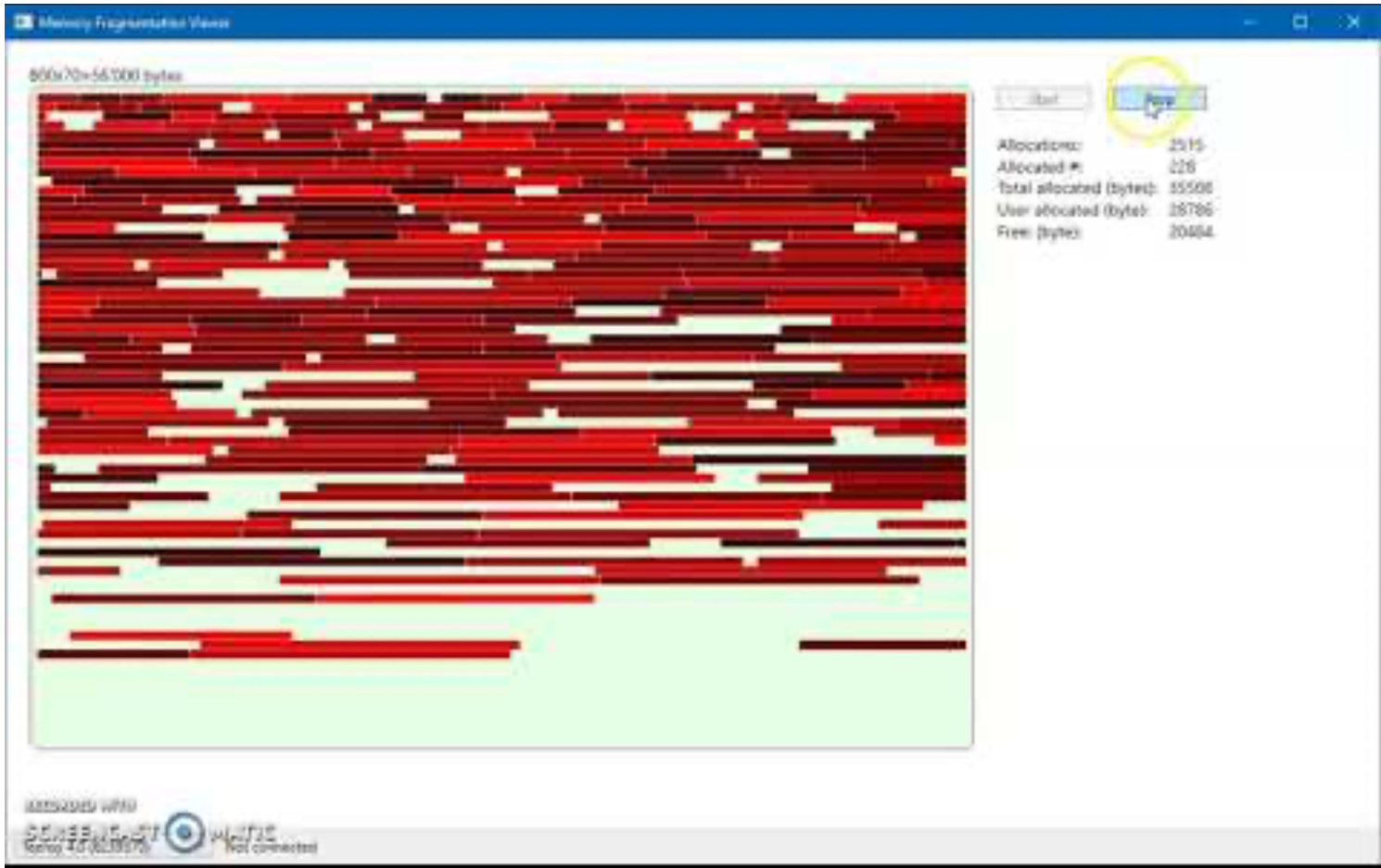
Add | Delete

```
/* Definitions for D3_Task */
osThreadId_t D3_TaskHandle;
const osThreadAttr_t D3_Task_attributes = {
  .name = "D3_Task",
  .stack_size = 128 * 4,
  .priority = (osPriority_t) osPriorityLow,
};
```


Only for use by students at BYUI currently enrolled in ECEN-361

Heap Management by FreeRTOS

As Tasks Come/Go – What happens to the Heap Memory?
"Fragmentation"



BYU IDAHO

Only for use by students at BYUI currently enrolled in ECEN-361

Summary of Memory Use / Access in FreeRTOS

- Heap Allocation Strategies
  - heap_1: Fixed-size allocation, no freeing; simplest but inflexible.
  - heap_2: Supports free and allocate but can lead to fragmentation.
  - heap_3: Wraps the standard library malloc and free functions.
  - heap_4: Uses a "best-fit" allocation strategy, minimizing fragmentation.
  - heap_5: Similar to heap_4 but supports memory regions for allocation.
- Garbage Collection?
  Garbage collection is an automatic memory management process used in software development to identify and reclaim memory that is no longer in use by a program

  NOT available in FreeRTOS. Manage your own.
- Memory Leaks?
  A memory leak in an (RTOS) occurs when a program or process fails to release memory that it has dynamically allocated but no longer needs.

BYU IDAHO

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

IDE's

- How am I developing the project's code?

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361

IDE's for the STM32

© Brigham Young University—Idaho

Only for use by students at BYUI currently enrolled in ECEN-361